



BigData

**An-Najah National University
Engineering and Information Technology**

**Final Project
Opportunity Matcher Project**

**Created by:
Aprar Ismail
Saja Dwaikat
Adham lahlooh
Ahmad Istatieh**

**Under Supervision:
Dr. Hamed Abdelhaq**

Academic Year: 2025 / 2026

Opportunity Matcher

1. Project Overview

- The Opportunity Matcher platform is a smart system designed to match students with suitable opportunities such as internships, projects, or job positions based on their skills, completed courses, and GPA.
- The system is built using a Big Data architecture to efficiently handle large-scale data and perform similarity matching in a scalable manner

Technologies used:

- **Backend:** Scala, Apache Spark, Apache Kafka, MongoDB, Akka HTTP
- **Frontend:** HTML, CSS, JavaScript
- **Matching Algorithm:** Locality Sensitive Hashing (LSH) to efficiently estimate similarity between student profiles and opportunities

2. Project Structure and File Description

A. Producer – KafkaProducerData.scala

Location: Producer

Why it is used:

- The producer is responsible for ingesting raw student data into the system in a streaming manner. Kafka is used to enable scalable and continuous data ingestion.

What it does:

- Reads student data from a CSV file and sends it as JSON messages to a Kafka topic.

How it works:

- Read the students_CVs.csv file.
- Converts the skills and courses columns into arrays to support later similarity matching.
- Sends data to the Kafka topic Data in batches of 500 records.
- Tracks the last sent record ID using last_sent_id.txt to avoid duplicate processing

Relation to other components:

- The streamed data is consumed by the Kafka consumer for further processing and storage in MongoDB

B. Consumer – kafkaConsumer.scala

Location: Consumer/kafkaConsumer.scala

Why it is used:

- To process incoming streaming data and prepare it for storage and analysis using Spark Structured Streaming.

What it does:

- Consumes messages from Kafka, cleans the data, and transforms it into a structured format.

How it works:

- Reads data from Kafka topic Data using Spark Structured Streaming.
- Parses JSON messages into a Spark DataFrame.
- Removes unnecessary columns such as gender, address, and birth_date.
- Converts date fields into proper date formats.

Relation to other components:

- This file is used by main.scala to build the complete streaming pipeline, and the processed data is passed to MongoWriter for storage.

C. Consumer – main.scala

Location: Consumer/main.scala

Why it is used:

- Acts as the main entry point for running the streaming pipeline.

What it does:

- Coordinates data ingestion, processing, and storage.

How it works:

- Creates a SparkSession.
- Configures Kafka and MongoDB connection settings.
- Calls kafkaConsumer.readAndCleanStream to read and clean the data stream.
- Writes processed data to MongoDB using MongoWriter.writeBatch.

Relation to other components:

- Depends on kafkaConsumer.scala and MongoWriter.scala.

D. Consumer – MongoWriter.scala

Location: Consumer/MongoWriter.scala

Why it is used:

- To efficiently store processed streaming data in MongoDB while avoiding duplication.

What it does:

- Writes Spark DataFrames into MongoDB.

How it works:

- Uses Spark MongoDB connector.
- Supports upsert functionality using:

- replaceDocument = true
- upsert = true
- Updates existing documents or inserts new ones as needed.

Relation to other components:

- Called directly by main.scala

E. Matching – LSHMatcher.scala

Location: Matching/LSHMatcher.scala

Why it is used:

- Traditional similarity matching is computationally expensive. LSH is used to efficiently estimate similarity at scale.

What it does:

- Implements the matching logic between students and opportunities using LSH.

How it works:

- Loads student data from MongoDB.
- Converts skills, courses, and major into tokenized feature vectors.
- Uses HashingTF and MinHashLSH to compute similarity scores.
- Returns the top 5 most similar students for a given opportunity.

Relation to other components:

- Used by LSHApiMain.scala to generate matching results.

F. Matching – LSHApiMain.scala

Location: Matching/LSHApiMain.scala

Why it is used:

- Provides an API layer to expose the matching functionality to the frontend.

What it does:

- Runs an HTTP server using Akka HTTP to handle matching requests.

How it works:

- Accepts POST requests at /match_students.
- Parses incoming JSON into an OpportunityReq object.
- Calls LSHMatcher.top5ForOpportunity to retrieve matching students.
- Returns results as JSON.
- Supports CORS to allow frontend access.

Relation to other components:

- Connects frontend requests with backend matching logic.

G. Frontend – HTML, CSS, JavaScript

- Files: index.html, styles.css, app.js

Why it is used:

- To provide a simple and interactive user interface for the system.

What it does:

- Allows users to upload opportunity data and view matching results.

How it works:

- Uploads a JSON file describing an opportunity.
- Converts file content into a request payload.
- Sends POST requests to /match_students.
- Displays top matching students with name, email, major, skills, and similarity score.

Relation to other components:

- Communicates directly with the API server implemented in LSHApiMain.scala.

3. Data Flow Overview

- The producer reads student CSV data and sends JSON messages to Kafka.
- Consumer pipeline (Kafka Consumer + Spark + MongoWriter) processes and stores data in MongoDB
- API Server receives opportunity requests and triggers similarity matching.
- Frontend sends requests and displays matching results to users

4. Key Points

- Each file has a clear and well defined responsibility.
- Kafka enables real time streaming of student data.
- Spark ensures scalable processing and transformation.
- MongoDB provides flexible and efficient data storage.
- LSH (MinHash) enables fast and scalable similarity matching.
- The system architecture supports scalability and real time matching.